LeetCode 80: Remove Duplicates from Sorted Array II

Before Starting: What Is This Problem Asking?

Do not think about code first. First understand the story.

You are given a sorted list of numbers:

`[1,1,1,2,2,3]`

Sorted means the same numbers stay together. For example, all the `1`'s are next to each other.

The rule is:

Each number is allowed to appear at most two times.

So:

- One `1` is okay.
- Two `1`'s are okay.
- Three `1`'s are not okay.

Therefore:

`[1,1,1,2,2,3]`

should become:

`[1,1,2,2,3]`

We removed only the extra third `1`.

Very Important: We Are Not Really Deleting

In this LeetCode problem, we do not need to truly delete elements from the array.

Instead, we rewrite the front part of the same array.

Example:

`nums = [1,1,1,2,2,3]`

After solving, the array may look like this:

`[1,1,2,2,3,3]`

This looks strange because there is still an extra `3` at the end.

But LeetCode only checks the first `k` elements.
Here the correct part is:

`[1,1,2,2,3]`

So we return:

$$k = 5$$

That means:

Only the first 5 numbers matter. The rest is ignored.

Beginner Way to Think About It

Imagine you are copying good numbers into the front of the same array.
We use a variable called `k`.

`k` means: where should I put the next good number?

At the beginning:

$$k = 0$$

because the next good number should be placed at index `0`.
Whenever we keep a number:

1. Put that number at `nums[k]`.
2. Move `k` one step forward.

So `k` also becomes the length of the valid answer.

The Main Question We Ask Every Time

For each number, we ask:

Can I keep this number, or would it become the third copy?

If it would become the third copy, we skip it.

The Simple Rule

The optimized code uses this rule:

$$k < 2 \quad \text{or} \quad num \neq nums[k-2]$$

This may look scary, so let us translate it into simple English.

Part 1: Why `k < 2`?

If we have kept fewer than two numbers so far, we can always keep the current number.

Why?

Because it is impossible to have three duplicates when we have not even kept two numbers yet.

So the first two numbers are always safe.

Part 2: Why Check `nums[k - 2]`?

This is the most important idea.

Before placing a new number at position `k`, we look two places back:

`nums[k - 2]`

If the current number is equal to `nums[k - 2]`, then adding it would make three copies.

Example:

Suppose the valid part is:

`[1,1]`

Now `k = 2`. The next place to write is index `2`.

If the current number is `1`, then compare it with `nums[k - 2]`:

`nums[0] = 1`

Current number is also `1`.

So if we keep it, the valid part becomes:

`[1,1,1]`

That is three `1`'s, so we skip it.

But if the current number is `2`, then:

`2 != nums[0]`

So it is safe to keep `2`.

Final Optimized Code

```python
class Solution:
    def removeDuplicates(self, nums: list[int]) -> int:
        k = 0

        for num in nums:
            if k < 2 or num != nums[k - 2]:
                nums[k] = num
                k += 1

        return k
```

Line-by-Line Explanation

Line 1

```
k = 0
```

This means the valid answer is empty right now.
It also means the next kept number should be written at index `0`.

Line 2

```
for num in nums:
```

This looks at every number from left to right.
For example, if:

`nums = [1,1,1,2,2,3]`

then `num` will become:

`1, then 1, then 1, then 2, then 2, then 3`

Line 3

```
if k < 2 or num != nums[k - 2]:
```

This line decides whether to keep `num`.
We keep it if one of these is true:

- We have kept fewer than two numbers so far.
- The current number is different from the number two positions behind.

Line 4

```
nums[k] = num
```

This writes the good number into the front part of the array.

Line 5

```
k += 1
```

This moves `k` to the next empty position.

Line 6

```
return k
```

This returns the length of the valid part.

Slow Dry Run

Input:

$$\texttt{nums = [1,1,1,2,2,3]}$$

At first:

$$k = 0$$

Current num	k before	Decision	Valid front part
1	0	Keep, because k ¡ 2	[1]
1	1	Keep, because k ¡ 2	[1,1]
1	2	Skip, because nums[k - 2] is 1	[1,1]
2	2	Keep, because 2 is not nums[0]	[1,1,2]
2	3	Keep, because 2 is not nums[1]	[1,1,2,2]
3	4	Keep, because 3 is not nums[2]	[1,1,2,2,3]

Final answer:

$$k = 5$$

Valid part:

$$\texttt{[1,1,2,2,3]}$$

Another Example

Input:

$$\texttt{nums = [0,0,1,1,1,1,2,3,3]}$$

Allowed result:

$$\texttt{[0,0,1,1,2,3,3]}$$

Why?

- Two `0`'s are allowed.
- Two `1`'s are allowed, but the third and fourth `1` are skipped.
- One `2` is allowed.
- Two `3`'s are allowed.

So the returned length is:

$$k = 7$$

Why This Works Only Because the Array Is Sorted

The array is sorted, so equal numbers are next to each other.

That means if a number appears three times, they will appear together like this:

`[1,1,1]`

So checking two positions behind is enough.

If the array was not sorted, this trick would not be safe.

Most Beginner-Friendly Summary

Remember only this:

We build the correct answer at the front of the same array.

`k` is the next place where we write a kept number.

Before keeping a number, compare it with the number two places behind.

If they are the same, keeping it would create three copies, so skip it.

If they are different, keep it.

Complexity

- Time complexity: $O(n)$, because we look at each number once.
- Space complexity: $O(1)$, because we do not create another array.